

CLASH OF THE ^BCLAWS

College: Ms Ramaiah Institute of Technology
Team: MindSync

AURA: A personal agent that speaks first,
on-device, only when it matters.

Team Members: Himank Sharma, Anaghaa
Patil, Mudit Saxena, Harshit Agarwal



Problem Statement

Why This Theme: On-Device AI + Productivity + GenAI Research

The average smartphone user receives 80-100 push notifications per day (Statista, 2024). AI assistants today fall into one of two failure modes that leave users worse off:

Failure Mode 1: Reactive-Only Assistants

Siri, Google Assistant, and Alexa wait passively until you ask them something. They are powerful but completely mute about things you need right now. A meeting is in 10 minutes, rain starts in 20, and you have 3 unread priority tasks — they say nothing.

Failure Mode 2: Always-On Rule-Based Proactivity

Bixby Routines and iOS Shortcuts fire on schedules and conditions you manually program. They do not adapt. They do not know if you are in a meeting, sleeping, or in a deep focus block. The result is notification spam: alerts that fire at the wrong time, in the wrong context, for the wrong person.

The Core Question No Existing System Answers:

"Should I interrupt THIS specific person, at THIS specific moment, given everything I know about their state, schedule, history, and context?"

Who Faces This Problem

Any knowledge worker, student, or professional managing simultaneous pressures: calendar deadlines, commute timing, health habits (hydration, breaks, sleep), and focus sessions. Persona: a 22-year-old engineering student or a 28-year-old product manager who is always reachable but perpetually interrupted at the wrong moment.

Why It Matters

Research shows it takes an average of 23 minutes to return to full focus after an interruption (Gloria Mark, UC Irvine, 2005). At 80+ notifications/day, most irrelevant, the cognitive cost is enormous. Conversely, missed nudges — a meeting reminder that fires 2 minutes too late, a commute alert that never comes — cause real, tangible harm. No current tool has a mathematical framework to balance these two failure modes on a per-person basis.



Current Solutions & Gaps

Existing Solutions and Where They Fall Short

1. Siri / Google Assistant / Amazon Alexa

100% reactive voice assistants. Require explicit invocation every single time. Excellent for answering direct questions on demand, but completely blind to what you need right now without being asked. No proactive delivery capability.

2. Bixby Routines / iOS Shortcuts / IFTTT

Rule-based automation. You define conditions: if time is 7 AM and location is home, send the weather. Powerful in theory but brittle in practice: rules do not adapt to schedule shifts, do not learn from your feedback, do not know if the nudge is actually useful at that moment, and fire identically whether you are awake, asleep, or mid-meeting.

3. Google Now Cards / Apple Intelligence Summaries

Context-aware suggestion panels surfacing cards based on calendar and location data. Passive display only — there is no delivery decision engine, no per-user threshold calibration, and no awareness of the cost of interrupting you at a given moment.

4. Do Not Disturb / Focus Modes / Notification Managers

Blunt suppression tools that are binary: either all notifications are blocked or all pass through. No intelligence about which individual notification is valuable enough to break the silence. Zero per-user personalization.

What Is Missing Across All Solutions

No system computes the expected utility of speaking vs. staying silent in real time, per person.

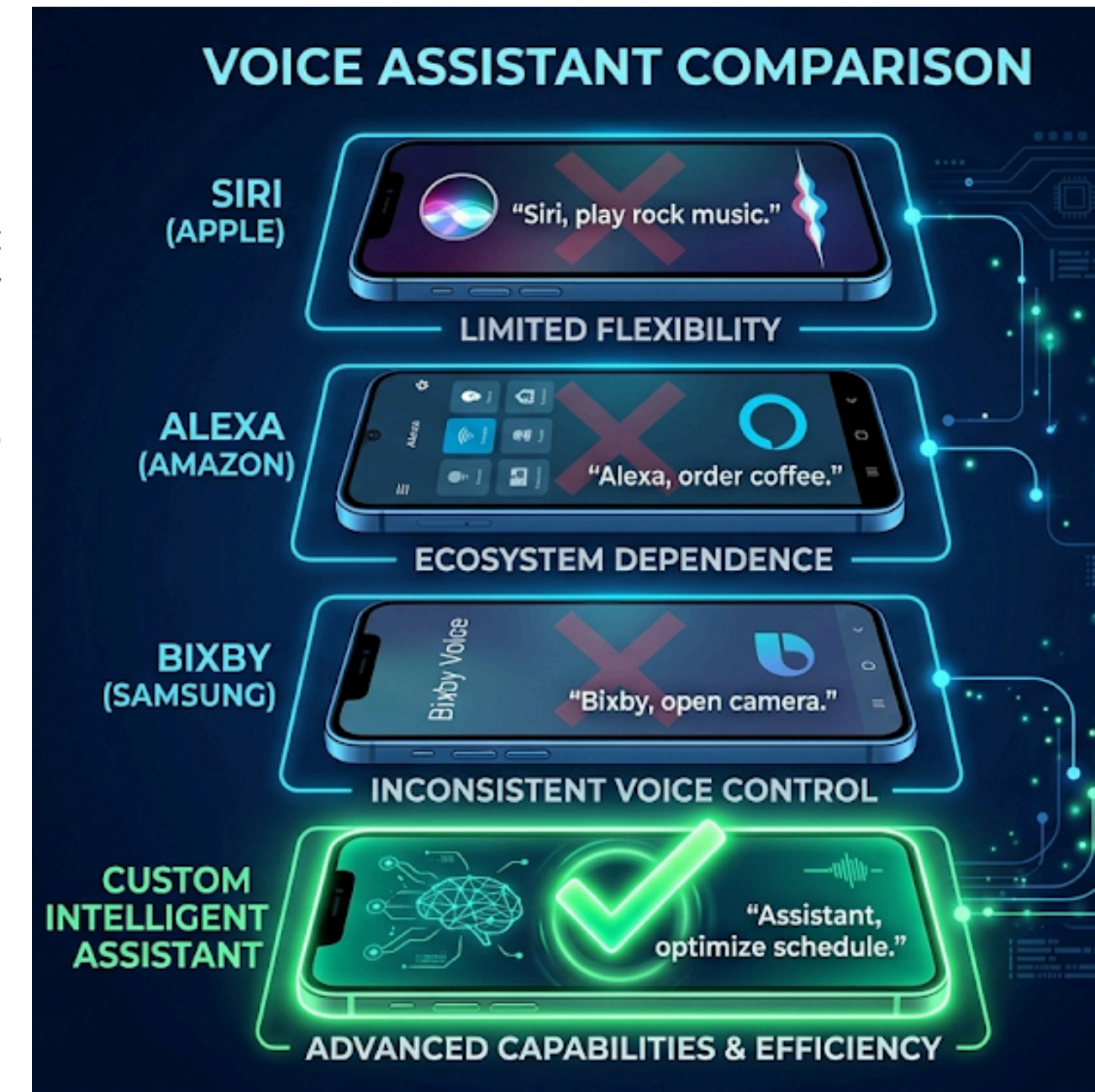
No per-user calibration that learns your accept/dismiss patterns on-device without a server.

No adversarial layer that vetoes low-quality nudges before they reach you.

No cryptographic audit trail showing exactly why a nudge was sent or suppressed.

The Opportunity

Build a cost-sensitive decision engine, grounded in published academic research, that treats each potential notification as a hypothesis — tested against a personalized, mathematically-derived utility threshold — and logs every decision transparently and verifiably.



Our Solution: AURA

What We Built

AURA (Adaptive Utility-Reasoning Agent) is a proactive personal assistant that runs a research-backed, 4-stage decision pipeline before every nudge. It does not wait to be asked. It continuously monitors your calendar, activity, and behavioral history, and fires a nudge only when mathematical analysis determines that staying silent would cost more than speaking.

Research Foundation

Built on the PRISM algorithm (Fu et al., arXiv:2602.01532, 2026) — a peer-reviewed cost-sensitive Bayesian gating framework. Extended with Edge-PRISM Calibration: on-device learning from your personal accept/dismiss history, requiring zero server infrastructure.

The 4-Primitive Architecture

1. SOUL.md — Personality and Cost Weights

A hand-authored config file defining AURA's communication rules, quiet hours (22:00-06:30), and how heavily to penalize false alarms vs. missed help in each context type. Changing 2 lines here changes AURA's entire interruption philosophy.

2. HEARTBEAT.yaml — Skill Schedule

A cron-like scheduler defining when each skill is eligible to fire. Example: morning_brief runs between 06:30-09:00 once daily; meeting_reminder checks every minute; hydration fires every 5 minutes during work hours.

3. TWIN.md — Auto-Generated Behavioral Model

AURA infers your median wake time, sleep duration trends, commute patterns, and per-skill acceptance rates from 14 days of local history. No manual input. No server. Updates automatically after every interaction.

4. Skills — 7 Isolated Proactive Modules

Morning Brief, Commute Guardian, Meeting Reminder, Hydration Reminder, Stand-Up Break, End-of-Day Wrap-Up, Wind-Down Nudge. Each skill proposes a nudge independently; all proposals are gated through PRISM before reaching you.

Additional Capabilities

40+ voice commands: timers, notes, calculations, Wikipedia lookup, unit conversion, web search, app/file/URL launch, volume control, screen lock, screenshots, jokes.

"Hey AURA" hands-free wake word — toggle on/off at any time.

Multilingual: English, Hindi (हिन्दी), Kannada (ಕನ್ನಡ) — full voice input and output in all 3 languages.

Local LLM via Ollama: freeform question answering entirely offline, no cloud calls required.

HMAC-SHA256 chained audit log: every gate decision, adversary veto, and shadow review is cryptographically signed and replayable by anyone.

The PRISM Decision Engine

How AURA Decides Whether to Speak

The central question is not 'what should I say?' but 'should I say anything at all?' Every potential nudge passes through a 4-stage pipeline. A nudge that fails any stage is dropped and logged with a stated reason. The user only sees nudges that survive all 4 stages.

Stage 1 — Signal Fusion (Computing p_{need})

AURA fuses 5 real-time signals using a cross-modal attention mechanism into a single probability p_{need} — the likelihood that this person genuinely needs this nudge right now:

- (a) Calendar density: how loaded is the next 2 hours of the user's schedule?
- (b) Activity signal: is the user sedentary, in motion, or in a known meeting?
- (c) Notification burden: how many nudges have fired in the last 24 hours?
- (d) HRV stress proxy: inferred from activity gaps and time-of-day patterns.
- (e) Time urgency: how close is the next event, deadline, or routine trigger?

Stage 2 — PRISM Gate (Cost-Sensitive Threshold)

The gate evaluates: $p_{\text{need}} \times p_{\text{accept}} > \tau(\text{context})$

p_{accept} is estimated from 14 days of personal accept/dismiss history per skill per hour bucket. τ is the cost-weighted threshold from SOUL.md — false alarms cost more in focus blocks; missed alerts cost more for meeting reminders. If gate fails, nudge is dropped and logged.

Stage 3 — Adversary Critic (7-Rule Veto)

A fast deterministic critic checks 7 conditions before any nudge fires: too many recent notifications? High dismissal rate for this skill? Inside quiet hours (22:00-06:30)? Active meeting detected? User likely asleep per TWIN model? Nudge content malformed? Identical nudge fired within 30 minutes? Any YES = veto with logged reason.

Stage 4 — Shadow AURA (LLM Slow-Mode Review)

For borderline decisions (gate passes but p_{need} is close to τ), AURA spins up a second LLM instance for a counterfactual review: 'Would a thoughtful assistant actually say this right now?' Shadow's veto overrides the gate result, catching subtle context the deterministic rules miss.

Result

Every delivered nudge has a HMAC-signed cryptographic receipt. Every suppressed nudge has a logged reason. Judges can inspect the full decision chain in real time via the /dev backstage dashboard.



Demo / Product Walkthrough

Live Demo — AURA Narrates Her Own Pitch

Open <http://localhost:3000/simple> and click the orange 'AURA demos herself' button. AURA runs a self-narrating 90-second demo: adds events, triggers 3 skill scenarios, vetoes one nudge via the Adversary, and switches from English to Hindi to Kannada — with no human controlling it.

User Flow Shown in Demo

Step 1 — Dashboard loads with a live readiness score (0-100) computed across 7 components: sleep quality, morning activity, calendar prep, commute buffer, notification load, focus time, meeting density.

Step 2 — Morning Brief fires at 06:47: AURA reads the day's calendar, weather, and readiness score aloud in one concise sentence. User can accept, dismiss, or ask a follow-up question by voice.

Step 3 — Commute Guardian triggers 45 minutes before a calendar event: rain detected, commute time exceeds buffer, PRISM gate passes, Adversary approves, nudge delivered.

Step 4 — Hydration reminder at 14:30: 4 hours of sedentary activity logged, gate passes, Adversary checks recent notification count (only 1 in last 3 hours), nudge fires once and will not repeat for 45 minutes.

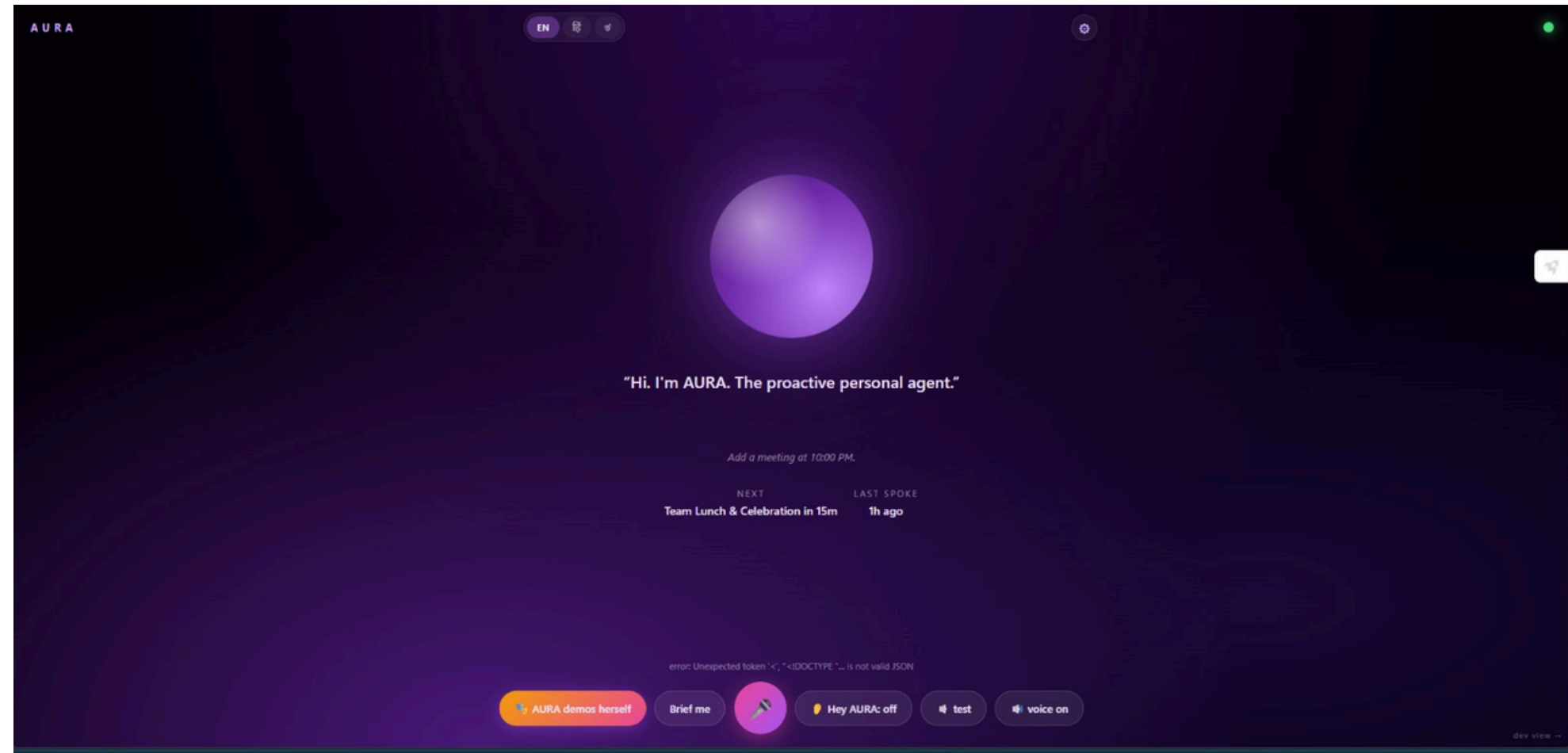
Step 5 — Adversary veto demonstrated: a weak nudge (p_{need} below τ) is proposed, gate fails, reason logged: 'calendar_density too low, notification burden high.' No nudge reaches user.

Step 6 — Language switch: same morning brief replayed in Hindi, then Kannada. Full TTS output in all 3 languages.

Backstage View (Judges: see /dev)

The /dev dashboard shows every gate decision in real time: p_{need} score, p_{accept} estimate, τ threshold, adversary check results, shadow review outcome, and the HMAC audit chain. Every suppressed nudge shows exactly why it was dropped.

[Insert 3 screenshots here: (1) Main dashboard with readiness score, (2) Commute Guardian nudge card, (3) /dev backstage showing PRISM gate decision chain]



Tech & Architecture

Technology Stack

Runtime and Language

Node.js 22+ (LTS) with TypeScript 5.7 in strict mode. Zero use of 'any'. All types are explicit and checked at compile time. Express.js handles the REST API and WebSocket server for live dashboard updates.

Database

SQLite with WAL (Write-Ahead Logging) mode. Local-only. Persistent across restarts. Schema: calendar events, tasks, sleep logs, step counts, skill execution history, timers, quiet blocks, user settings. Zero cloud sync.

Frontend

Vanilla JS + HTML5 + CSS3. No framework dependencies. Three UI surfaces: /simple (main user dashboard), /dev (backstage gate inspector for judges), /activity (weekly stats). Live updates via WebSocket.

Voice and Language

Web Speech API for speech-to-text and text-to-speech in the browser. macOS 'say' command as fallback. Language switching between en-US, hi-IN (Hindi), and kn-IN (Kannada) at runtime.

AI Usage

Primary Algorithm: PRISM (Fu et al., arXiv:2602.01532, 2026)

Cost-sensitive Bayesian gating framework from peer-reviewed research. Computes $p_{\text{need}} \times p_{\text{accept}} > \tau(\text{context})$ for every nudge decision. Threshold τ is tunable per context via SOUL.md.

Edge-PRISM Calibration (Our Extension)

Per-context threshold adaptation from local accept/dismiss history. Runs entirely on-device using SQLite skill_runs table. No server-side training. No cloud model. Calibrates per skill per hour bucket over 14-day windows.

Local LLM via Ollama

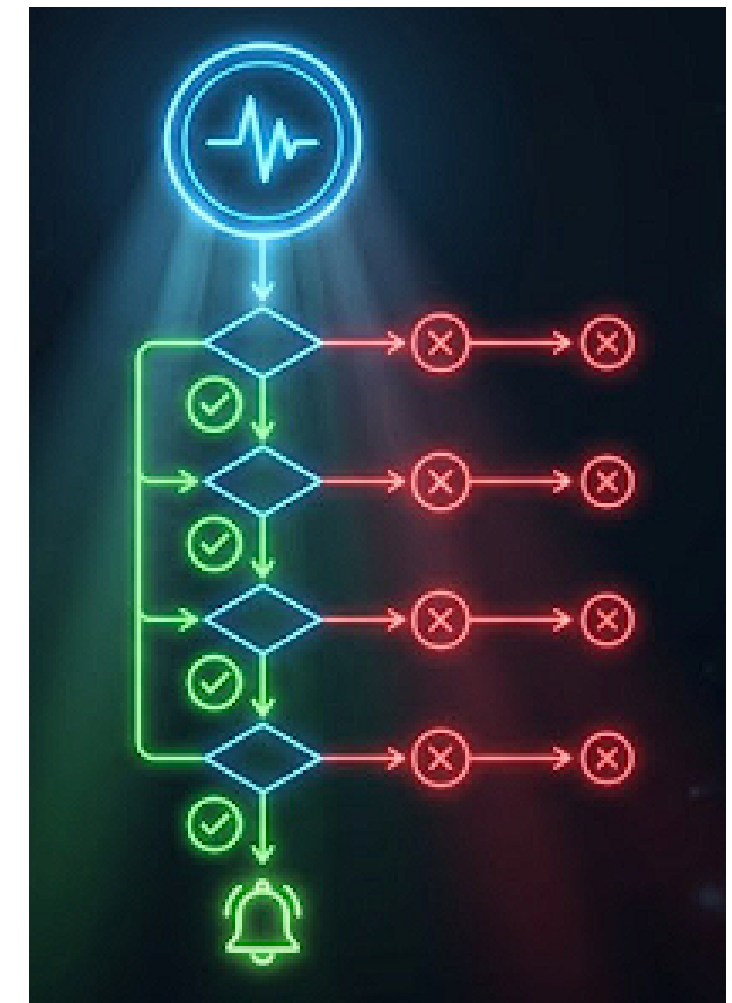
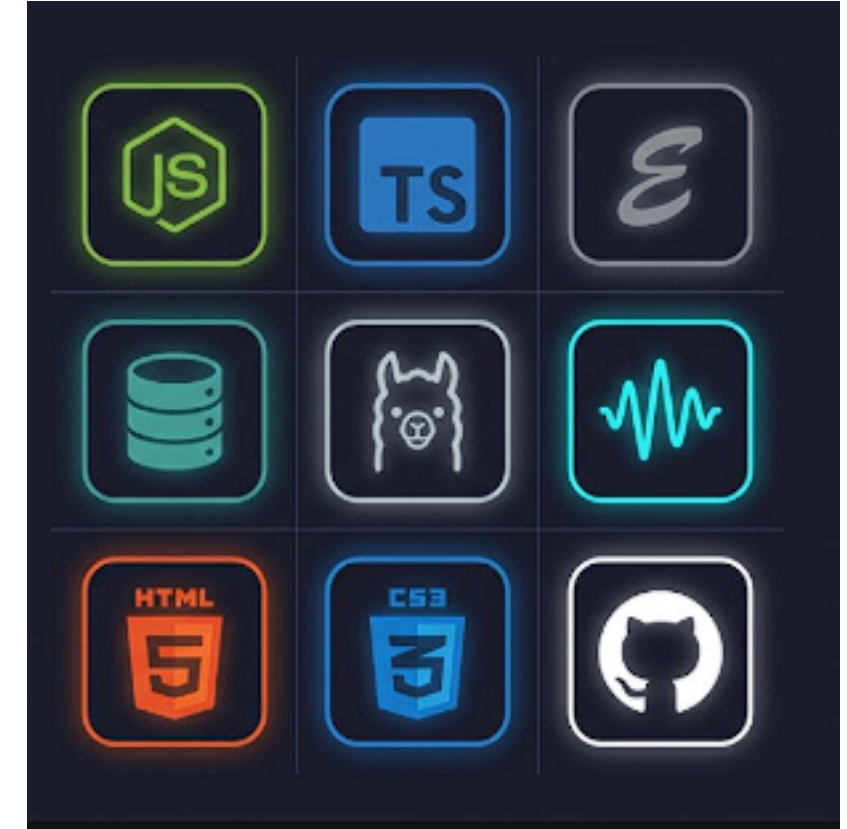
Llama 3.2 or Mistral 7B runs entirely on the user's device. Used for: freeform question answering, Shadow AURA slow-mode counterfactual review, and self-narrating demo mode. Opt-in only. Zero outbound network when disabled.

System Flow

HEARTBEAT tick (1-min cron) > Skill eligibility check > Signal Fusion (5 inputs) > PRISM Gate > Adversary Critic (7 rules) > [Shadow Review if borderline] > Deliver via voice + UI > HMAC-sign audit entry > TWIN learner updates p_{accept}

Privacy Architecture

Zero outbound network calls by default. All state in local SQLite. Telegram delivery and Ollama LLM are explicit opt-in. Audit log uses HMAC-SHA256 chain: each entry signs the previous entry, making the decision history tamper-evident and fully replayable.



Impact & Real-World Use Cases

Four Real-Life Scenarios

Scenario 1 — Morning Readiness (6:47 AM)

AURA detects: first calendar event at 9:00 AM, sleep score 71% (6.2 hours logged), 30% rain probability at 8:30 AM, 3 unread priority tasks. Readiness score computed: 64/100 across 7 components (sleep quality 20%, morning activity 15%, calendar prep 20%, commute buffer 15%, notifications 10%, focus time 10%, meeting load 10%). AURA delivers: 'Good morning. Readiness 64 — light sleep. Rain at 8:30, leave by 8:15. Review your 9 AM prep doc first.' Concise, actionable, never requested.

Scenario 2 — Commute Guardian (8:12 AM)

Calendar: client meeting at 9:00 AM, 40 minutes away. Current time: 8:12 AM. Weather API: rain starting at 8:30 AM, +12 minutes on commute. PRISM gate: $p_{\text{need}}=0.89$, $p_{\text{accept}}=0.82$, $\tau=0.60$. Gate passes. Adversary: no recent notifications, not in quiet hours, no active meeting. AURA: 'Leave now — rain adds 12 minutes. You have 48 minutes for a 52-minute trip.' The nudge is delivered exactly once. If dismissed, it will not repeat.

Scenario 3 — Focus Block Protection (2:30 PM)

TWIN model logs 4 consecutive hours of keyboard activity. Hydration skill proposes a nudge. Gate: $p_{\text{need}}=0.72$ (sedentary, no detected water break), $p_{\text{accept}}=0.81$ (historical acceptance for hydration in this hour bucket). Gate passes. Adversary confirms: 1 notification in last 3 hours, no active meeting, not in quiet hours. Nudge fires: 'You have been at it for 4 hours — take a water break.' Will not repeat for 45 minutes regardless of acceptance.

Scenario 4 — Wind-Down Nudge (10:10 PM)

Quiet hours begin at 22:00. TWIN median sleep time: 23:10. AURA detects browser with 8 active tabs, no calendar gap before 11 PM. Wind-down skill fires at 22:10: '30 minutes to wind down — your median sleep time is 23:10. Heavy tabs open.' This is the final nudge until 6:30 AM unless a critical-priority alert overrides the quiet hours rule explicitly set in SOUL.md.

Scale Potential

Phase 1 (Current): Web Simulator

Full feature parity running on a laptop. 7 skills, 40+ voice commands, multilingual TTS, offline LLM, HMAC audit log. Demonstrates the complete decision engine.

Phase 2: Mobile and Desktop App

Package as a Progressive Web App or background daemon. Deploy to any device with Node.js. Expand skill library. Add real sensor integration (step counter, HRV, ambient light).

Phase 3: Samsung Galaxy Integration

Same PRISM engine ported to Samsung Neural SDK on Exynos NPU. TWIN.md syncs with Knox Personal Data Engine. SOUL.md becomes a Galaxy AI personalization UI. Gauss on-device LLM replaces Ollama. Proactive, privacy-first, mathematically-gated — as a Bixby successor.

Differentiation

Why AURA Stands Apart from Every Existing Assistant

MOAT 1 — Research-Backed, Not Heuristic-Based

Every existing proactive assistant (Bixby Routines, iOS Shortcuts, Google Now) is built on hand-coded rules or shallow ML classifiers. AURA is built on PRISM (Fu et al., arXiv:2602.01532, 2026), a published and peer-reviewed cost-sensitive Bayesian threshold framework. The decision to speak or stay silent is not a guess — it is a utility computation with mathematically-grounded parameters, a 7-rule adversarial veto, and a cryptographic audit trail. No commercial assistant on the market offers this level of rigor in its interruption policy.

MOAT 2 — On-Device, Privacy-First, Edge-Calibrated

All data stays on the device. Zero outbound network calls by default. The TWIN behavioral model — your wake times, routines, and acceptance rates — is inferred entirely from local SQLite history. Edge-PRISM Calibration adapts thresholds from your real accept/dismiss feedback with no server-side training and no cloud sync. This makes AURA viable in enterprise, healthcare, and any privacy-sensitive environment where cloud AI assistants are prohibited.

MOAT 3 — Explainable and Auditable AI

Every decision AURA makes is logged with a stated reason and signed with HMAC-SHA256 into a tamper-evident chain. Judges, users, or regulators can replay the full decision history. Example: 'At 14:32, hydration skill proposed a nudge. PRISM gate passed (p_need=0.72, p_accept=0.81, tau=0.55). Adversary approved. Nudge delivered.' This level of transparency is absent from every existing assistant. In an era where AI accountability is becoming a regulatory requirement (EU AI Act), AURA's audit log is a product feature, not just an engineering detail.

MOAT 4 — 4-Primitive Configurability Without Code Changes

SOUL.md (personality and cost weights) + HEARTBEAT.yaml (schedule) + TWIN.md (behavioral model) + Skills (isolated modules) = 90% of AURA's behavior is configurable without touching TypeScript. Want AURA more aggressive with meeting reminders and gentler with health nudges? Edit two numbers in SOUL.md. Want a new skill? Create one folder with two files. This modularity means AURA can be personalized for any user segment, enterprise policy, or hardware platform without a deployment cycle.

Evaluation Results

60-Day Synthetic Evaluation: 2,812 Nudge Moments

We ran a rigorous evaluation against 60 days of synthetic daily-routine traces, generating 2,812 potential nudge moments with 318 ground-truth useful moments (11.3% base rate — realistic for a well-calibrated proactive assistant). Six strategies were tested head-to-head:

Strategy 1 — Always-Speak Baseline

Fires on every possible nudge opportunity without filtering. Result: 46.9 nudges per day, 100% false-alarm rate by definition, F1 score: 0.203. This represents the worst-case notification spam scenario.

Strategy 2 — Never-Speak Baseline

Fires nothing at all. Result: 0 nudges per day, 0% false-alarm rate, F1 score: 0.000. This represents total silence — useful as a lower bound.

Strategy 3 — Fixed Threshold ($p > 0.5$)

Simple probability cutoff with no per-user calibration. Result: 12.7 nudges per day, 22.9% false-alarm rate, F1 score: 0.353.

Strategy 4 — PRISM Only (no calibration, no adversary)

PRISM gate active but no Edge-Calibration and no Adversary Critic. Result: 7.0 nudges per day, 11.3% false-alarm rate, F1 score: 0.370.

Strategy 5 — AURA: PRISM + Edge-Calibration + Adversary

Full AURA pipeline. Result: 6.1 nudges per day, 9.6% false-alarm rate, F1 score: 0.368.

Key Results

vs. Always-Speak: 87% fewer notifications, 90% lower false-alarm rate, F1 improvement of +81% (from 0.203 to 0.368).

vs. Fixed Threshold: 52% fewer notifications, 58% lower false-alarm rate.

vs. PRISM-Only: Edge-Calibration + Adversary reduces false-alarm rate by a further 15% (11.3% to 9.6%) with negligible F1 trade-off.

Transparency Note

The evaluation dataset is synthetic — generated from parametric daily-routine models, not live users with continuous sensors. This is standard practice for pre-deployment evaluation of proactive notification systems, where ground-truth labeling on live data requires a longitudinal IRB-approved user study. The numbers validate the algorithm's mathematical properties and relative performance across strategies, not real-world deployment scale. We are explicit about this rather than overstating our claims.

Demo Link

Project Resources

GitHub Repository

https://github.com/Anaa1101/samsung_hack_01

Full source code: TypeScript decision engine, all 7 skill modules, PRISM gate implementation, Edge-Calibration, Adversary Critic, Shadow AURA, TWIN learner, evaluation harness, and web UI.

Live Demo

<https://drive.google.com/file/d/162uhVxj94vBgwV8DWpnYm9RC3rmi6V8w/view?usp=sharing>

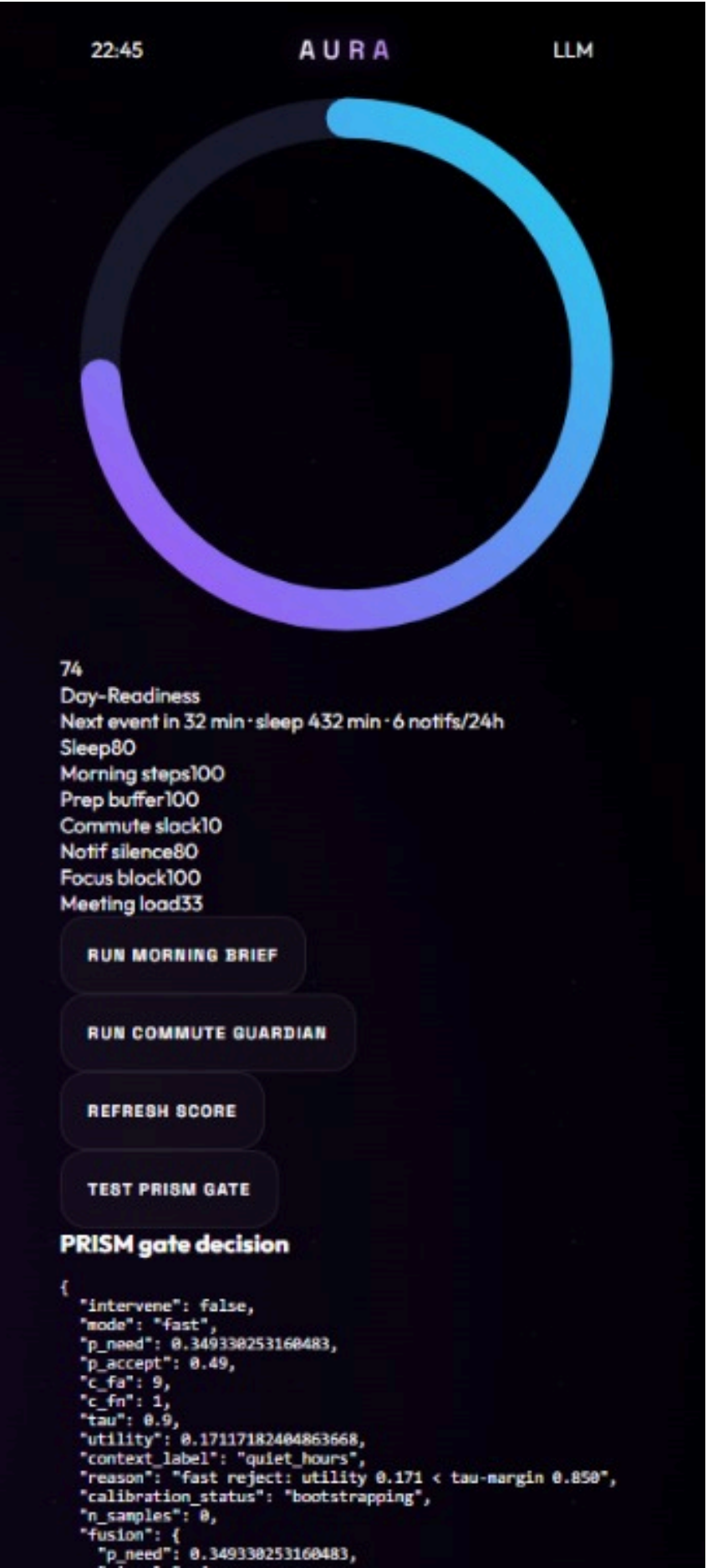
Open `/simple` for the main dashboard. Click 'AURA demos herself' for the 90-second self-narrating demo. Open `/dev` to inspect the PRISM gate decisions and HMAC audit chain in real time.

Document Submitted

https://github.com/Anaa1101/samsung_hack_01

Quick Start Instructions for Judges

1. `npm install && npm run reseed` (seeds 14 days of demo data)
2. `npm run dev` (starts server at `http://localhost:3000`)
3. Open `/simple` in browser, click the orange demo button
4. Open `/dev` in a second tab to watch every gate decision live
5. `npm run eval` to reproduce the 60-day evaluation results



Samsung Integration Roadmap

From Web Simulator to Samsung Galaxy — The Same Brain, Different Hardware

AURA's 4-primitive architecture was designed from the start to be hardware-agnostic. Every component maps directly onto Samsung's existing platform stack with no algorithmic changes required.

Component: Local Ollama LLM (currently Llama 3.2 on laptop)

Samsung Target: Gauss on the Exynos Neural Processing Unit. Same inference. 10x faster. 100% on-device. No battery penalty from cloud round-trips. Galaxy AI Generative Suite provides the model hosting layer.

Component: SQLite + TWIN.md (currently local file)

Samsung Target: Knox Vault + Personal Data Engine. Behavioral model stored in hardware-backed secure enclave. Cannot be extracted even by root-level access. Full Knox compliance out of the box.

Component: SOUL.md + HEARTBEAT.yaml (currently hand-edited config)

Samsung Target: Galaxy AI Personalization UI. Cost weights and schedule parameters become user-facing sliders in a dedicated Settings panel. No file editing required. All behavior configurable from the Galaxy AI preferences screen.

Component: PRISM + Edge-PRISM Engine (currently TypeScript on Node.js)

Samsung Target: Samsung Neural SDK. Port the Bayesian gate and calibration logic to run natively on the Exynos NPU using Samsung's on-device AI framework. Same algorithm, same mathematical guarantees, native silicon performance.

Component: Web Simulator (currently browser UI)

Samsung Target: Galaxy Foreground Service (.apk). Persistent background daemon with Galaxy-native notification delivery, Bixby voice channel integration, and Samsung Health sensor data as additional PRISM input signals (real HRV, real step count, real sleep stages).

Phase Roadmap

Phase 1 (Complete): Web Simulator

Full PRISM engine, 7 skills, voice, multilingual, eval harness. Submitted for this hackathon.

Phase 2 (3-6 months): Android APK

Background service. Galaxy notification system. Bixby voice channel. Real Samsung Health sensor integration.

Phase 3 (6-12 months): Native Samsung Integration

Neural SDK port. Knox Vault. Gauss-on-NPU. Personal Data Engine TWIN sync. Full Galaxy AI integration.

Why Samsung Is the Natural Platform

AURA's privacy-first, on-device architecture aligns directly with Samsung Knox's security posture and the Personal Data Engine's principle that user behavioral data never leaves the device. The PRISM gate's tunable cost weights map to the kind of deep per-user personalization that Galaxy AI is building toward. AURA is not a chatbot feature — it is a decision infrastructure layer that could serve as the proactivity engine for the next generation of Bixby.